Mobile Vehicle Counter Server

This is the Autonomous Systems Inc.'s project to count vehicles, and analyze traffic through a subdivision gate.

This specific project is the Server. It accepts POSTED data from deployed raspberry pi 5 units, and serves data to an angular frontend to report data to analysts.

We separate this into two servers so that we can keep revising the server for the frontend (which we expect to need changes), while keeping the data receiving server constantly running:

1. one listens to events from the deployed raspberry pi 5 units, and writes these to the sqlite database.
2. the other reads from the database and publishes the data to the angular frontend.

Two Servers

This project creates two server binaries (in `src/bin/`), both thin wrappers around a `run_server()` in the library crate:

Binary	Calls	Default address	Purpose
`run_data_collector`	`collector_server::run_server()`	`127.0.0.1:3001` (`COLLECTOR_ADDR`)	Receives POSTED base64 events, saves images to disk, inserts rows, and acknowledges each with a SHA-256 receipt; safe to retry (see "Retries and duplicates"). **Creates and migrates the database if it is missing.** Pi-side contract: `README_PI.md`.
`run_frontend_api`	`api_server::run_server()`	`127.0.0.1:3000` (`API_ADDR`)	Backend for the angular frontend (login, locations, event stats). **Fails at startup if the database is missing.**

`default-run` in `Cargo.toml` is `run_frontend_api`, so a bare `cargo run` starts the **API**, not the collector. Use `cargo run --bin run_data_collector` for the collector.

Throughout this document, "the collector" means `run_data_collector` and "the API" means `run_frontend_api`.

There is also `migrate_data` (`cargo run --bin migrate_data`), a **one-off** tool that copies legacy data from `out/alpr_events.db` into `server_db/server.db`. It hard-codes both paths relative to the current directory and seeds a legacy test location (`TCEV`), sublocation (`Balimbing`) and two cameras. It is not part of normal operation.

Design Decisions (read before changing things)

Two processes, one shared SQLite file

- The collector and the API are **separate processes that share one SQLite file**. They share no code paths at runtime and have **separate state structs** (`CollectorState` vs `handlers::app_state::AppState`) on purpose: only the collector writes image files.
- The reason for the split is operational: the API will be redeployed/restarted often; the collector should stay up so no Pi events are lost. **Restarting the API must never require restarting the collector.**
- The collector is the main writer; the API mostly reads (but it also writes: `users`, `location`, `sublocation` CRUD). Concurrent access relies on SQLite's locking. Keep write transactions short.
- Both servers bind to `127.0.0.1` only. They are meant to sit behind nginx.

Collector: reverse proxy and route

- The collector listens on `127.0.0.1:3001` (not 3000; 3000 is the API). nginx needs a `proxy_pass` to it.
- The route is **`POST /api/alpr/event`** (singular `event`).
- Payload (JSON): `event_id`, `camid`, `date`, `plate`, `plate_image`, `image`. `image` and `plate_image` are base64 (standard alphabet) JPEGs. `date` must be **RFC 3339 with offset**, e.g. `2026-10-09T16:44:25.443+08:00`. `event_id` is a **random (version 4) UUID** generated by the Pi (see "Retries and duplicates" below).
- **The full Pi-side contract (request, response, status codes, retry rules, examples) is in `README_PI.md`.** Keep it in step with any change to the collector's request or response.
- The request body is limited to **20 MB** (`MAX_BODY_BYTES` in `src/collector_server.rs`); anything larger gets `413`. axum's own default is only 2 MB, which would reject large scene images (base64 adds about a third). The nginx `client_max_body_size` in `deploy/nginx-collector.conf` must be at least as large.
- Images are saved as `event_<uuid>.jpg` and `event_<uuid>_plate.jpg`. Only the **filenames** are stored in the DB, not paths or bytes. The directory is not configured per-row, so the API/frontend must know where the images directory is.

Collector: validation order and responses

The handler deliberately validates cheaply first and does expensive work last, and cleans up after itself:

Step	Failure	Response
0	(The SHA-256 of the raw body is computed; cannot fail)	
1	Body is not valid JSON / fields missing (including `event_id`)	`400`
1	`event_id` is not a UUID	`400`
2	`date` not RFC 3339	`400`
3	`camid` not in the `camera` table	`422` (**unknown cameras are rejected**; cameras must be registered first)
3	DB error looking up camera	`500`
4	`event_id` already stored, **same** hash	**`200`**, `duplicate: true` (no work done, nothing written)
4	`event_id` already stored, **different** hash (or stored row has no hash)	`409`
4	DB error looking up the event	`500`
5	Base64 decode / disk write fails	`500` (a half-written scene image is deleted)
6	DB insert hits a UNIQUE constraint (a concurrent retry won)	Same answer as step 4. **The image files are kept** (they are the winner's).
6	Any other DB insert failure	`500` (both image files are deleted)
done	Event stored	**`200`**, JSON body `{"event_id", "content_hash", "duplicate": false}` (not `201`, see below)

Success bodies are JSON (`Content-Type: application/json`); **error bodies are plain text** (e.g. `Invalid JSON payload`, `Unknown camera 'X'`). A `GET` on the route returns `405`, and any other path returns `404`.

`200` is used for both a first delivery and a duplicate, rather than `201 Created` for a first delivery. A retry is not a creation, and this way the Pi has a single success code to check; `duplicate` in the body says which happened.

Retries and duplicates

The Pi may re-POST an event (its connection can drop at any point, including after the server stored the event but before the reply arrived). The protocol makes this safe:

- **Identity: `event_id`.** A **version 4 (random) UUID** generated by the Pi, once per event, reused on every retry. It is the primary key of `alpr_events` and is part of the image filenames, so it is validated as a UUID (this also rules out path traversal) and stored lowercase-hyphenated.
    - **Why v4 specifically:** the whole duplicate-detection scheme assumes two different events never get the same `event_id`. With genuinely random v4 UUIDs (122 random bits) that is not a practical concern: for example 500 Pis recording 5,000 events a day for 10 years (about 9 billion events) have roughly a 1 in 10^17 chance of producing even one repeated id. The assumption fails only through poor randomness or non-random ids, which is why other versions are ruled out: v1/v6 use a clock and MAC address (Pis cloned from one SD card image can share them), v3/v5 hash a name so equal input gives an equal id, and v7 is time-ordered with less randomness. Ids must also never be derived from the plate, the camera, the time or a counter.
    - **The server does not enforce the version.** It only checks that the value parses as a UUID (any version), so a non-v4 id is accepted, and nothing on the server flags it. The v4 rule is a requirement on the Pi software, written down in README_PI.md ("The `event_id`"), which has Rust guidance (`Uuid::new_v4()`). This is deliberate: rejecting by version would turn a harmless difference into a `400`, which the Pi's retry rules classify as "never retry", so the event would be set aside. If you want it enforced anyway, check `uuid.get_version_num() == 4` where `event_id` is parsed in `process_event()`, and update the `400` row above and the tests.
- **Receipt: `content_hash`.** The server computes the **SHA-256 (lowercase hex) of the raw request body bytes**, before parsing, stores it in `alpr_events.content_hash` (nullable, `UNIQUE`), and returns it. The Pi hashes the bytes it sent; the event is delivered only if the hashes match. The raw bytes are hashed (not the parsed JSON) so no JSON canonicalisation has to be agreed between the two sides. The consequence is that **the Pi must resend the exact same bytes** on a retry.
- **Why both, rather than the hash alone:** two genuine events can have identical content, and a pure content hash would drop one of them. The `event_id` decides what is a duplicate; the hash is the integrity check (and detects an id reused for different data).
- **Race safety:** uniqueness is guaranteed by the database (the primary key / the `UNIQUE` index), not by the "already stored?" lookup in step 4, which is only a shortcut that saves decoding and rewriting the images. Two simultaneous retries can both pass the lookup; one insert then fails with a unique violation and is answered as a duplicate. This is covered by an automated concurrency test.
- **A duplicate must never touch the original's files.** The image filenames derive from `event_id`, so a duplicate would write to the same names. Step 4 returns before any write, and the insert-conflict path deliberately does **not** delete files (the cleanup in the `500` path would otherwise delete the original's images).

- **A rejected attempt does not reserve the `event_id`.** E.g. a 422 for an unregistered camera leaves nothing behind, so once the camera is registered the same bytes succeed.
- **Rows without a hash** (stored before the protocol, e.g. by `migrate_data`) have `content_hash` NULL. The unique index allows any number of NULLs. Such a row can never be reported as a duplicate: if a Pi reuses its id the answer is `409`, since its content cannot be compared.
- `content_hash` is **not** included in the frontend API's JSON (`#[serde(skip)]` on the entity field). It belongs to the Pi protocol. If `src/entities/alpr_events.rs` is regenerated with `sea-orm-codegen`, re-apply that attribute and the field, or the frontend response will change (or fail to build).
- The migration `m20261030_000001_add_content_hash` adds the column and the unique index. It is applied automatically at collector startup, and was tested on a copy of the existing database (existing rows kept, hash NULL) and rolled back with `down`.
- `camid` is matched against `camera.id` in exactly one place, `find_camera()`. If the device-reported id ever stops being the camera primary key, change only that function.
- Files are written **before** the DB insert, and removed if the insert fails, so a failure should not leave orphaned images. A crash between the two steps still could.

Dates and time zones

- `alpr_events.date` is `timestamp_with_time_zone`, but SQLite has no time zone type: the value is stored as **RFC 3339 text**, and the range filter (`start <= date < end`) and the ordering in `/api/time_limits` are done by the database **comparing that text character by character**. That only matches time order when every stored value and every query bound uses the **same offset**.
- **Everything is normalised to `+08:00` (Philippine time, no DST).** The whole system runs in one time zone, so this is deliberate and lives in one place, `src/time.rs`:
  - The **collector** converts the Pi's `date` to `+08:00` before saving, whatever offset the Pi sent (`Z`, `-05:00`, ...). The instant is unchanged; only the offset used to write it down changes. The Pi's original offset is **not** kept.
  - The **API** converts `start_datetime` / `end_datetime` to `+08:00` for the SQL filter in `/api/binned_events` and `/api/bin_events_details`. The frontend may send any offset (e.g. UTC from `toISOString()`).
  - Bins and the `start_time` / `end_time` of `/api/binned_events` are returned in the offset the frontend asked for.
  - Verified: with events sent as `+08:00`, `Z` and `-05:00`, the same window written as `17:00+08:00`-`18:00+08:00` and as `09:00Z`-`10:00Z` returns identical events, and UTC-hour and `+08:00`-hour bins give identical counts.
- **Sub-second digits are not fixed-width** (SeaORM writes none for a whole second, otherwise three, e.g. `...:25+08:00` and `...:25.443+08:00`). That is fine: with one offset, text order still equals time order (tested), so no custom column type is needed. The collector keeps millisecond precision (anything finer is dropped when the value is written).
- **Rows that bypass the collector must also be `+08:00`.** `migrate_data` copies legacy dates as-is (the legacy file was checked: all rows are `...+08:00`), and rows inserted by hand into SQLite are not normalised. A row stored with another offset would be mis-ordered and mis-filtered again.
- `/api/time_limits` returns the stored value, so it is returned as `+08:00`.
- If the deployment ever spans more than one time zone, revisit this properly (see `TODO.md`).
- Time ranges are **half-open**: `start <= date < end`.

Binned events (`/api/binned_events`)

- Binning is done **in Rust, not SQL**: all events in the range are loaded into memory, empty bins are generated, then each event is assigned to a bin (a linear scan per event). Simple and fine at current volumes; revisit (SQL `GROUP BY` or a sorted/index-based assignment) if ranges get large (e.g. months of minute-bins).
- The last bin is truncated at `end_datetime`.
- Month bins advance by calendar month and clamp the day (Jan 31 -> Feb 28/29). Note that clamping is applied to the *running* start, so after clamping, later bins keep the clamped day (Jan 31 -> Feb 28 -> Mar 28).
- `advance_time` has a `panic!` that should be unreachable; an enormous bin count (e.g. `minutes` over years) is the more realistic risk.
- Events are matched to sublocations through `camera.sublocation_id` (inner join), so an event only appears if its camera belongs to one of the requested sublocations.

Data model

`location` -> `sublocation` -> `camera` -> `alpr_events`. All primary keys are **strings** (UUIDs for API-created rows; the legacy migration uses readable ids like `TCEV`; `alpr_events.id` is the **Pi-supplied `event_id`**). `users.id` is an integer.

Relationship	On delete	On update	Consequence
`sublocation` -> `location`	CASCADE	CASCADE	Deleting a location deletes its sublocations.
`camera` -> `sublocation`	RESTRICT	CASCADE	A sublocation with cameras **cannot be deleted** (API returns 500, not 409).
`alpr_events` -> `camera`	RESTRICT	RESTRICT	A camera with events cannot be deleted or re-id'd.

- Because of the above, deleting a **location** that contains a sublocation with cameras will fail (the cascade hits the `camera` RESTRICT). Events are effectively permanent.
- `camera.orientation` has a CHECK constraint: only `Inbound` or `Outbound`.

- There is **no API to create cameras**. Cameras are inserted by hand (directly in SQLite, e.g. `INSERT INTO camera(id, name, description, sublocation_id, orientation) VALUES (...)`, after creating the `location` and `sublocation` rows) or by the legacy `migrate_data` tool, and the collector rejects events from unregistered cameras. `camera.id` must equal the `camid` that the Pi sends.
- `location.description` is required (`NOT NULL`); `sublocation.description` is optional.
- SQLite foreign keys are enforced by sqlx by default; do not disable them.
- The entities in `src/entities/` are `sea-orm-codegen` output ("@generated"). Prefer regenerating from the DB over hand-editing.

Authentication

- `POST /api/login` is public; **every other API route is behind `auth_middleware`** (JWT Bearer, HS256, secret from `JWT_SECRET`). `route_layer` is used so unmatched paths still 404 rather than 401.
- Tokens expire in **24 hours** (`expires_in: 86400`). There is no refresh token and no revocation; `is_active = false` only blocks *new* logins, not already-issued tokens.
- Passwords are hashed with **Argon2** (PHC string in `users.encrypted_password`). Login returns `401` for unknown user, inactive user, bad hash and wrong password alike (no user enumeration).
- **The `users` table is a placeholder.** It exists so the frontend has a login to show and so earlier code could be reused. **Who the users will be, and how roles and permissions work, is undecided (TBD).** Today there are no roles: any valid login can call every API route, and there is no API to create or change users.
- `users.email` has **no UNIQUE constraint**. Duplicate emails are accepted by the database (verified), and login would then pick one of the rows arbitrarily. Nothing in the API creates users, so this only matters if rows are inserted by hand. Add a unique index when the user model is designed.
- `JWT_SECRET` is read from the environment **on every request**, not at startup. If it is missing, the server starts fine and then returns `500` on every login/authenticated request.
- **The collector endpoint has no authentication in the Rust code.** The shared-secret check is done by nginx instead: `deploy/nginx-collector.conf` requires an `X-Collector-Token` header and answers `401` otherwise, so the collector must only be reachable through nginx (keep `COLLECTOR_ADDR` on `127.0.0.1`). See "Deployment" below.
- The first migration seeds a user **`admin@admin`** / **`asdf`**. Change or remove it in any real deployment. (The integration tests log in with it.)
- CORS is `allow_origin(Any)` on both servers and only allows `GET`/`POST` (note: the API also has `DELETE` routes, which a browser preflight would block cross-origin; this is fine only while the frontend is served same-origin via nginx).

API conventions

- Most "query" endpoints are `POST` with a JSON body (`/api/time_limits`, `/api/binned_events`, `/api/bin_events_details`) because they take lists of sublocation ids.
- Handlers return bare `StatusCode`s on error (no JSON error body) and log details with `eprintln!`. Success codes: `201` create, `204` delete, `404` when deleting a missing id.
- **Exception: errors raised by axum itself carry a plain-text body.** A request body that is not valid for the endpoint (a missing field such as `description`, or an unknown `interval` such as `years`) returns **`422`** with axum's message, e.g. `Failed to deserialize the JSON body ... missing field `description``. Handler errors never have a body.
- **Database constraint failures are `500`, not `4xx`.** This covers deleting a sublocation that has cameras, deleting a location whose sublocations have cameras, **and creating a sublocation with a `location_id` that does not exist** (foreign key violation). The cause is only in the server log.
- An end time before the start time in `/api/binned_events` is not an error: it returns `200` with `[]`.

Configuration

Loaded with `dotenvy` from `.env` in the **current working directory**.

Variable	Used by	Notes
`DATABASE_URL`	both servers, tests	**Required** (the process panics at startup if absent). Use `sqlite://server_db/server.db?mode=rw`. Relative paths resolve against the cwd.
`JWT_SECRET`	the API, tests	Required at request time (see Authentication).
`API_ADDR`	the API	Listen address. Default `127.0.0.1:3000`. Keep it on `127.0.0.1` (the API is meant to sit behind nginx), and update the nginx `proxy_pass` if you change the port.
`COLLECTOR_ADDR`	the collector	Default `127.0.0.1:3001`.
`IMAGE_DIR`	the collector	Default `images`. Created automatically if missing.

Gotchas: - The images directory variable is **`IMAGE_DIR`** (`.env` used to say `IMAGES`, which the code never read). - **`dotenvy` does not override variables already set in the shell.** A stale `DATABASE_URL` exported in your shell (e.g. from another project) silently wins over `.env`. If the

server complains about an unrelated database driver, run `unset DATABASE_URL`. - **Do not put `journal_mode=WAL` (or any other pragma) in `DATABASE_URL`.** sqlx 0.9's SQLite URL parser only accepts `mode`, `cache`, `immutable` and `vfs`; any other parameter is a hard error (`unknown query parameter 'journal_mode'`) and the server refuses to start. SQLite pragmas are set in code instead, see "SQLite connection settings" below. - `DATABASE_URL` should use `mode=rw`. File creation is opt-in in code (`MissingDatabase::CreateAndMigrate`), used only by the collector; the API fails fast on a missing file. - Everything is relative to the working directory (`.env`, `server_db/`, `images/`). Run both servers from the project root (or use absolute paths / systemd `WorkingDirectory`).

SQLite connection settings (WAL)

Both servers open the database through **`db::connect(url, MissingDatabase)`** (`src/db.rs`), never `Database::connect` directly. This keeps the two processes configured identically (only the missing-database behavior differs). Every connection is opened with:

Setting	Value	Why
`journal_mode`	`WAL`	The collector writes while the API reads. In SQLite's default rollback-journal mode a writer blocks readers (`database is locked`); in WAL readers and the single writer do not block each other.
`synchronous`	`NORMAL`	The usual WAL companion. Safe against an application crash; a power loss / OS crash can lose the most recent commits, but cannot corrupt the database.
`busy_timeout`	5 s	Writers wait for the write lock instead of failing immediately. Only matters when the API writes (users/locations/sublocations) at the same moment as the collector.
`foreign_keys`	`ON`	The schema relies on its RESTRICT/CASCADE rules.

Decisions and reminders: - WAL is set **on every connection** rather than once with `PRAGMA journal_mode=WAL` on the file. It is persistent in the file, but doing it in code means a freshly migrated or restored database cannot silently be left in rollback-journal mode. - Verified: with a write transaction held open, the API still answers (login `200` in ~0.5 s); the same scenario in rollback-journal mode gives `database is locked`. - WAL needs both processes on **the same machine** (it uses shared memory, `server.db-shm`). Do **not** put the database on NFS/SMB. - Expect `server.db-wal` and `server.db-shm` next to `server.db`. Do not delete them while a server is running. - **Backups:** copying only `server.db` can miss committed data still in the `-wal` file. Use `sqlite3 server_db/server.db ".backup backup.db"` (or copy all three files). - There is still only **one writer at a time**. If collector write volume ever outgrows that, the next step is a client/server database (e.g. Postgres), not more SQLite tuning. - SeaORM defaults a SQLite pool to **one connection** when `max_connections` is not set, and `db::connect` does not change that. Concurrency between the two *processes* comes from WAL; within one process, queries are serialised through that single connection. Raise it in `db::connect` if the API becomes a bottleneck. - The integration tests in `tests/` still call `Database::connect` directly (plain rollback-journal behaviour unless the file is already in WAL). That is fine for them, but they do not exercise `db::connect` (tracked in `TODO.md`).

The database file and what happens when it is missing

The schema is owned by the `migration/` crate (SeaORM migrations). Whether a missing database is created is decided **in code**, via the `MissingDatabase` argument of `db::connect()` (`src/db.rs`), not by `mode=` in `DATABASE_URL`:

Server	`MissingDatabase`	Missing file
`run_data_collector`	`CreateAndMigrate`	Creates the parent directory and the file, then runs all pending migrations (`Migrator::up`). Also applies pending migrations to an existing/empty file.
`run_frontend_api`	`Fail`	Exits immediately with `unable to open database file`. It never creates or migrates.

The collector overrides the URL's `mode=rw` with `create_if_missing(true)`, so `.env` keeps `mode=rw` for everyone. Start the collector first on a fresh deployment; the API then finds a ready database. Auto-created databases contain the seeded `admin@admin` user from the migrations.

Why the collector creates the database: the "reset after testing" workflow

This is deliberate. Both servers run as long-lived services, and **exactly one collector instance runs at a time**. The collector is allowed to create the database so that, after a testing phase, you can wipe the test data and go live without running any migration command by hand:

1. Stop both services (the collector and the API).
2. Delete the database **and its WAL side files**: `rm -f server_db/server.db server_db/server.db-wal server_db/server.db-shm`. (If you forget the side files, the collector will refuse to start and tell you to delete them.)
3. Start the collector. It recreates the directory/file, migrates, and seeds `admin@admin`.
4. Start the API.

5. Re-create the `location` -> `sublocation` -> `camera` chain (there is no API to create cameras, and the collector rejects events from unregistered cameras with 422).
6. Change or remove the seeded `admin@admin` / `asdf` user.

Leftover files make the collector refuse to start. If the database file is missing but `server.db-wal` or `server.db-shm` still exist, the collector exits with an error naming the files and telling you to delete them and restart. A stale `-wal` next to a brand-new database can be replayed into it or confuse SQLite, so this is checked *before* anything is created. (Side files next to an existing database are normal and are not reported.)

Creating a new database is logged loudly. When the collector creates the file, it prints a `WARNING: database file not found. A NEW, EMPTY database was created and migrated at: <absolute path>` banner to stderr, so it is easy to spot in the service logs (e.g. `journalctl`). It is **not** printed on a normal restart. If you see it unexpectedly, `DATABASE_URL` probably points at the wrong path.

Because only one collector ever runs, there is no locking around the create-and-migrate step. Starting two collectors at once against a missing database can make one of them fail with `UNIQUE constraint failed: seaql_migrations.version`. That is not a supported setup.

The old `images/` directory (and `out/`) are **not** touched by this. Leftover images are orphaned (the DB no longer references them) but harmless, so clearing them is optional. Both directories are git-ignored on purpose.

What is (and is not) in git

- `server_db/` is **git-ignored**. The database is created at runtime by the collector, never committed. (It used to be tracked; it was removed from the index with `git rm --cached`. If you pull an old checkout that still tracks it, your local file is not deleted by that, but a checkout of an old commit can overwrite it, so back it up first.)
- `images/` and `out/` are git-ignored.
- Open items that were deliberately deferred are in `TODO.md`.

What actually happens

Verified by running the binaries with `DATABASE_URL=...?mode=rw`:

Situation	Result
Collector, **file and/or directory missing**	Creates directory + file, migrates, starts normally, and logs the `NEW, EMPTY database` warning.
Collector, file missing but **`-wal` / `-shm` left over**	**Exits immediately** with an error naming the leftover files. Delete them and restart.
Collector, existing database	Starts normally, applies any pending migrations, no warning.
API, **file missing**	**Exits immediately** with `unable to open database file` (code 14).
API, `mode=rwc` in the URL, file missing	**Still exits** with `unable to open database file`: the API always passes `create_if_missing(false)`, which overrides the URL. (Verified.)
Variable missing	Both servers **panic** at startup (`DATABASE_URL must be set in .env file`).
File deleted **while running**	Processes **stay alive**; requests return `500`. (On Linux the open handle keeps the old inode, so behavior can vary; restart after restoring.)
Tables missing (empty/unmigrated DB)	Processes **stay alive**; API returns `500` (`no such table: users`), collector returns `500` (`no such table: camera`). **Events from the Pis are rejected** until the DB is migrated.

Design implications / reminders: - The failure is **loud at startup only when the file cannot be opened**. The remaining silent case is a database that exists but has no tables (e.g. restored from a broken backup): the collector's startup migration normally fixes that, but if a migration fails midway the process exits rather than running half-migrated. Monitor for non-2xx from the collector (the Pis should retry/queue on failure). - Per-request DB errors never crash the processes. Handlers map DB errors to `500` and log them. - Neither server retries the connection: the connection is opened **once at startup**. If the DB is unavailable at boot, the process exits and must be restarted by a supervisor (systemd `Restart=on-failure`). - `.env` uses `mode=rw` so a missing/misplaced database fails fast in the API. Only the collector creates the file, and it does so in code. A typo in `DATABASE_URL` therefore creates a fresh database when starting the collector, so check the path if the collector unexpectedly starts with no data. - If both servers start at the same moment against a missing database, only the collector migrates; the API fails fast and can be restarted by its supervisor (`Restart=on-failure`). Start the collector first, or let the supervisor retry the API. - Because the two servers are independent, one being down does not affect the other, but they fail **together** if they share the same missing/broken database file.

Creating / migrating the database manually

Normally not needed (the collector does it). To do it by hand with the migration CLI:

```
mkdir -p server_db
cd migration
DATABASE_URL="sqlite://../server_db/server.db?mode=rwc" cargo run -- up
```

The migration CLI needs `mode=rwc` to create the file. (See `migration/README.md` for the other CLI commands. Note the path is relative to `migration/`, and the shell `DATABASE_URL` caveat above applies. The migration CLI uses its own plain connection, so it does not go through

db::connect.) The migrations create the tables and seed the admin@admin user. After migrating you must still insert at least one location -> sublocation -> camera chain before the collector will accept any event.

Deployment

Draft deployment files for the collector are in deploy/. Both are heavily commented; lines marked CHANGE need your values (domain name, paths, user).

File	Purpose
deploy/mobile-vehicle-collector.service	systemd unit that runs run_data_collector as a service (restart on failure, runs as an unprivileged user, read-only filesystem except server_db/ and images/).
deploy/nginx-collector.conf	nginx site: TLS, redirect from port 80, the X-Collector-Token shared-secret check, a 20 MB body limit, per-IP rate limit, and proxy_pass to 127.0.0.1:3001. Only POST /api/alpr/event is proxied; every other path is a 404.

The shared secret is **not** in the repo. nginx cannot read environment variables, so the config includes /etc/nginx/collector-token.conf, which you create on the server (the config's header comment shows the exact contents). The Pi sends the same value in an X-Collector-Token header on every POST. Because the secret travels in a header, it is only protected if the connection is HTTPS.

Both files were tested: the unit with systemd-analyze verify, and the nginx config by running it in front of the real collector (no/wrong token 401, correct token 200 and a stored row, GET 403, other paths 404, body over 20 MB 413). The nginx config avoids http2 on; so it works on nginx older than 1.25.1.

Checklist for a first deployment: 1. Build: cargo build --release --locked --bin run_data_collector. 2. Create the service user, .env (a fresh one, not the committed one), and install the unit. 3. Start the collector once so it creates and migrates the database, then register the location -> sublocation -> camera rows (the camera id must equal the Pi's camid). 4. Create the nginx token file, get a certificate (certbot), install the nginx config, and run nginx -t. 5. Test from the Pi (or curl) with the X-Collector-Token header.

Dependencies and Cargo.lock

- **Cargo.lock stays in git.** This crate builds binaries (the two servers), and for applications the Rust convention is to commit the lockfile so every build uses exactly the same dependency versions. Use cargo build --locked for deployment builds so a stale lockfile fails the build instead of being silently updated.
- migration/Cargo.lock is also committed, but it is only used when building the migration CLI from inside migration/. The main build ignores it and resolves the migration path dependency through the root Cargo.lock.
- **Build-time cost of the migration dependency:** the servers now depend on the migration crate (so the collector can migrate at startup), which pulls in sea-orm-migration and, through it, sea-orm-cli and clap. These are compiled into both binaries even though the API never migrates. This is accepted for now; if build time or binary size matters, make sea-orm-migration's CLI support optional.

Testing

- cargo test --lib runs the unit tests in src/db.rs (URL parsing and the leftover-file check). They need no database or .env.
- tests/location_test.rs and tests/sublocations_test.rs are **integration tests that run against the real database in DATABASE_URL** (not a temp/in-memory DB), log in as admin@admin/asdf, create rows, and delete them. They therefore need a migrated database (so start the collector once after a reset) and JWT_SECRET, and will leave rows behind if a test fails midway. Don't point them at production data.
- tests/collector_test.rs tests the collector's retry/duplicate protocol: first POST, retry (duplicate: true), conflicting content (409), invalid and missing event_id, existing rejections, files untouched by a retry, recovery after a simulated crash between writing the files and inserting the row, legacy NULL-hash rows, and 12 concurrent retries of one event (exactly one row and one pair of files, every reply 200). Unlike the other integration tests, each test builds its **own temporary database through db::connect** (WAL, migrations) and images directory under the system temp dir, so they need no .env or existing database and can run in parallel: cargo test --test collector_test. They call process_event() directly rather than going over HTTP, so the body-size limit, routing and nginx are **not** covered by them (those were checked by hand with curl). The tests were checked to be able to fail by temporarily breaking the conflict handling and the file cleanup.
- There are no automated tests for the events (statistics) endpoints of the API.
- http_tests/login.http is a manual request file for the IDE HTTP client (API on port 3000).

Repository notes

- .gitignore excludes target/, .idea/, images/, out/, migration/target and server_db/. **.env is not ignored and is currently tracked in git**, so JWT_SECRET is committed (see TODO.md). Consider ignoring it and committing a .env.example instead; rotate JWT_SECRET if this repo is ever shared.
- server_db/server.db was committed in earlier commits, so it is still present in **git history** (including the seeded admin user's hash), even though it is no longer tracked.
- out/ holds the legacy database (alpr_events.db) and legacy images, used only by migrate_data.